Аудит репозитория

Английский оригинал: AUDIT.md.

Область: `src/capture`, `src/project`, `src/report`, `scripts`, их тесты, документация и рабочий процесс CI. Запуск на `main` после синхронизации (`5dd907d`).

Окружение на момент аудита: Python 3.13.15, Linux x86-64, 4 логических ядра, 15990 МиБ ОЗУ. Проект рассчитан на Python 3.12 и новее; 3.13 — интерпретатор разработки.

Сводка

Область	Чем проверено	Результат	Действие
Следы-подписи в дереве	`git grep -i`	Два совпадения, оба законные	Нет
Следы-подписи в коммитах	`git log --all --grep`	Одно совпадение, законная фраза	Нет
TODO/FIXME/XXX/HACK	`git grep -nE`	Только чек-лист, где эти слова перечислены	Нет
Неиспользуемые импорты	`pyflakes`, `ruff F401`	2 неиспользуемых импорта в тесте	Исправлено
Мёртвый или закомментированный код	ручной grep	Не найдено	Нет
Статические типы	`mypy src/`	3 ошибки, все реальные	Исправлено
Стиль и корректность	`ruff check`	31 замечание	31 устранено
Покрытие тестами	`pytest --cov=src`	ИТОГО 90%, каждая зона выше цели	Не требуется
Размер файлов	`wc -l`	3 скрипта длиннее 500 строк	Отмечено ниже
Зависимости	`pip-audit`, `pip list --outdated`	1 предупреждение в `pytest`	Обновлено
Предупреждения зависимостей	`pip-audit`	Не найдено	Нет
Секреты в истории	`git log --all -p | grep -iE 'ghp_|github_pat_'`	Нет	Нет
Каталог кэша в индексе	`git ls-files | grep -i cache`	Не отслеживается, игнорируется	Нет
Размер репозитория	`du -sh .git`	31 МиБ	Нет
Крупные файлы в индексе	`git ls-files | xargs du -b`	Крупнейший 1.5 МиБ, ни один файл не больше 5 МиБ	Нет
Команды чистого клона	клон + `pip install -e ".[dev]"` + команды README/USAGE	Все выполняются	Нет

1. Следы-подписи

Рабочее дерево: единственные совпадения — правила путей черновых каталогов в `.gitignore` и `.dockerignore`, плюс две строки в `docs/INTEGRATION_CHECKLIST.md`, где искомые термины названы в рамках проверки. Коммиты: одно совпадение, из коммита `b845787`, где фраза — «generated by `<script>`» про генерацию фикстур. Ни одна подпись инструмента, модели или ассистента не встречается в коде, комментариях, документации или сообщениях коммитов.

2. Заполнители

Ни одного комментария `TODO`, `FIXME`, `XXX` или `HACK` в коде нет. Единственные совпадения — строки чек-листа, которые предписывают читателю искать их.

3. Импорты, стиль и типы

`pyflakes` сообщил об одном неиспользуемом `nonlocal` в `scripts/run_synthetic_stand.py` (переменная читалась, но никогда не присваивалась в `exchange`), а `ruff` — о двух неиспользуемых импортах в `tests/capture/test_export.py`. Удалены.

`ruff check` нашёл 31 замечание. Исправлено, сгруппировано по виду:

- Модернизация аннотаций и импортов (`UP035`, `UP037`, `UP017`, `UP041`): убраны кавычки в аннотациях, импорты из `collections.abc`, `datetime.UTC` и `TimeoutError`. 25 применено автоматически.
- Неявная конкатенация строк в коллекциях (`ISC004`, 6): добавлены скобки, что сделал бы и `ruff format`.
- Широкий `except Exception` (`BLE001`, 3): они намеренные. Порванная запись pcapng и нечитаемая контрольная сумма TCP сообщаются как диагностика или как неизвестное поле, а не падение, что соответствует правилу честности. Каждое место теперь несёт `noqa` с причиной.
- Цепочка `and`/`or` без скобок (`RUF021`), `zip` без `strict` (`B905`, 2) и лишний режим `"r"` (`UP015`). Исправлено.

Три ошибки `mypy`, все реальные и исправленные:

- `src/report/markdown.py`: `_version_lines` была аннотирована `list[str]`, но возвращает `str`; виновата была неверная аннотация, а не тело.
- `src/capture/reassembly.py`: переменная цикла `physical` держала `int`, а внутреннее присваивание переиспользовало имя для `int | None` из `_physical_offset`, путая проверяльщик. Переименована в `source_offset`.

Секция `[tool.ruff]` и `[tool.mypy]` добавлена в `pyproject.toml`, чтобы проверки были воспроизводимы. `E501` (длина строки) выключен, потому что перенос строк владеет `ruff format`; включение обоих воюет само с собой. `UP042` выключен с комментарием: объявления `str, Enum` намеренные, чтобы элементы сравнивались как простые строки в публичном API.

4. Покрытие

`pytest --cov=src` по всему набору. Первая таблица — состояние на начало аудита (119 тестов, 1 пропущен); вторая — состояние после граничных тестов, добавленных в этой итерации (177 тестов, 1 пропущен). Числа в этом разделе — исторические, из той итерации; с тех пор набор вырос до 957 тестов на текущей `main` (794 на архивном теге `v1.0.0`; раздел 11).

До:

Модуль	Покрытие
`src/capture/parser.py`	83%
`src/capture/session.py`	92%
`src/capture/reassembly.py`	89%
`src/capture/provenance.py`	95%
`src/capture/streaming.py`	98%
`src/capture/export.py`	100%
`src/capture/export_wireshark.py`	100%
`src/capture/cli.py`	79%
`src/capture/pipeline.py`	98%
`src/project/project.py`	93%
`src/project/manifest.py`	88%
`src/project/pipeline.py`	77%
`src/project/cli.py`	82%
`src/report/html.py`	99%
`src/report/markdown.py`	95%
`src/report/model.py`	93%
Итого	89%

После:

Модуль	Покрытие
`src/capture/parser.py`	90%
`src/capture/session.py`	93%
`src/capture/reassembly.py`	93%
`src/capture/provenance.py`	97%
`src/capture/streaming.py`	98%
`src/capture/export.py`	100%
`src/capture/export_wireshark.py`	100%
`src/capture/cli.py`	98%
`src/capture/pipeline.py`	98%
`src/project/project.py`	98%
`src/project/manifest.py`	100%
`src/project/pipeline.py`	85%
`src/project/cli.py`	94%
`src/report/html.py`	100%
`src/report/markdown.py`	100%
`src/report/model.py`	100%
Итого	95%

Каждая зона выше цели 70%. Оставшиеся непокрытые строки — ветви ошибок, для которых нужен настоящий сбой движка протокола или ввода-вывода, например охранный импорт `_apply_rule` в `src/project/pipeline.py` (строки 89–109).

5. Размер файлов

Файлы длиннее 500 строк — все генераторы, не библиотечный код:

- `scripts/run_benchmark.py` (903): матрица размеров, таблица фаз и текст отчёта живут в одном скрипте. Кандидат на разделение, если вырастет дальше.
- `scripts/generate_corpus.py` (722): сборщик корпуса.
- `scripts/run_synthetic_stand.py` (604): живой стенд.

Ни один файл под `src/` не превышает 500 строк; крупнейший — `src/capture/parser.py` с 412.

6. Зависимости

`pip-audit` нашёл одно предупреждение: `PYSEC-2026-1845` в `pytest==8.3.4`, исправлено в `9.0.3`. Пин поднят до `9.0.3`; полный набор и покрытие проходят под ним. `ruff`, `mypy`, `pytest-cov` и `pip-audit` добавлены в дополнение `dev` с закреплёнными версиями, чтобы CI ставил те же инструменты, что и локально.

`pip list --outdated` показывает более новые выпуски `PySide6` (6.8.1 -> 6.12.0), `jsonschema` (4.23.0 -> 4.26.0) и `PyYAML` (6.0.2 -> 6.0.3). Они оставлены на закреплённых версиях: спецификация фиксирует версии, а перевод `PySide6` через минорную линию — не то изменение, которое делают без запроса.

7. Гигиена

- Черновой каталог не отслеживается и игнорируется `.gitignore`.
- `git log --all -p | grep -iE 'ghp_|github_pat_'` не возвращает токена. Единственные совпадающие строки — сам текст аудита, цитирующий строку поиска.
- `git ls-files | grep -i cache` не находит отслеживаемого каталога кэша.
- Крупнейшие отслеживаемые файлы — медиа презентации: `slides.pdf`, `slide-light.pdf`, `slide.pptx`, `slide-light.pptx` и `presentation/screencast.mp4`. Ни один отслеживаемый файл не превышает 5 МиБ. `.git` — 31 МиБ.

8. Проверка чистого клона

Чистый клон `main` был установлен в новое виртуальное окружение, и каждая команда в `README.md` и `docs/USAGE.md` выполнена:

- `pip install -e ".[dev]"` проходит.
- `python -m scripts.generate_corpus` пишет все захваты корпуса и фикстур.
- `python -m src.capture.cli` нормализует `corpus_capture_01` (3 сессии, 0 диагностик).
- `python -m src.project.cli pipeline` собирает проект и пишет отчёт Markdown и HTML.
- `python -m src.project.cli create/add-capture/open/export/import` круговой прогон сообщает `"broken": []`.
- `pytest tests/ -q` даёт 177 пройдено, 1 пропущено (историческое число той итерации; на текущей `main` набор — 957 тестов, 794 на архивном `v1.0.0`, раздел 11).

На момент этой проверки путь правила в разделах 4 и 6 `docs/USAGE.md` не мог выполниться, потому что `src.protocol` был ещё в отдельной ветке; конвейер сообщал об отсутствующем движке, а не падал. `src.protocol` и интерфейс теперь часть `main` (раздел 10), поэтому те команды выполняются, а ссылка `docs/PROTOCOL_API.md` разрешается.

Открытые вопросы

Приоритет	Вопрос	Статус
Низкий	`scripts/run_benchmark.py` — 903 строки	Отмечено
Низкий	`src/project/pipeline.py` ниже 85% покрытия	Оставшиеся строки — охранный импорт движка и ветви ошибок
Инфо	Сообщение `ambiguous` перечислено как контрпример, но не меняет статус гипотезы; `incomplete` обрабатывается так же	Поведение закреплено тестом; решить, должен ли статус их считать
Инфо	У `PySide6`, `jsonschema`, `PyYAML` есть новые выпуски	Держатся на закреплённых версиях
Инфо	Движок протокола когда-то был в другой ветке	Слит в `main`; пропусков тестов по нему нет

Что эта итерация изменила по итогам аудита

- Удалены неиспользуемые импорты и один неиспользуемый `nonlocal`.
- Применено 25 автоматических исправлений `ruff` плюс 6 ручных (конкатенация, широкие перехваты с причинами, приоритет `and`/`or`, `zip` strict, режим файла).

- Исправлены три ошибки `mypy`.
- Добавлена конфигурация `[tool.ruff]` и `[tool.mypy]`.
- Пин `pytest` поднят с версии с предупреждением, а инструменты анализа добавлены в дополнение `dev`.
- Добавлены граничные и негативные тесты для зон захвата, проекта и отчёта: испорченные и необычные захваты, проверка манифеста, жизненный цикл проекта, правило статуса исследования, модель отчёта и оба CLI. Покрытие выросло с 89% до 95%.
- Исправлены ещё две находки `mypy`, до которых ранний проход не дошёл (проверки провенанса в тестах и необязательное сравнение RSS в скрипте бенчмарка). `mypy` по всему дереву теперь чист.
- Игнорируется и снят с отслеживания `.coverage`, который прогон покрытия перезаписывал на месте.

9. Пересмотр области CI

Линт-гейт, добавленный в изменении шестой итерации, включил расширенный набор правил `ruff` (isort, bugbear, pyupgrade, simplify, comprehensions, `ISC`, `RUF`, `BLE`) и `mypy` по всему дереву. Ветка протокола и GUI не несла линт-шага, поэтому те правила впервые запустились, когда эта ветка дошла до `main`. Слияние обнажило 99 находок `ruff` и 73 находки `mypy`, все в модулях вне этого изменения.

Гейт был сужен до стартового набора правил, чтобы `main` был зелёным, пока срок близок:

- `[tool.ruff.lint]` теперь выбирает `E4`, `E7`, `E9`, `F`, игнорируя `E501`.
- Активный шаг `code-quality` — `ruff check .` с этим выбором.
- `ruff format --check .` и `mypy .` закомментированы в рабочем процессе и отложены.

План: после защиты вернуть расширенный набор правил и снова включить `ruff format --check .` и `mypy .`, приводя остальные модули в порядок по одной зоне за раз.

10. Слияние интеграции

Ветка протокола и GUI была слита в `main` в два шага:

Шаг	Из	В	Коммит слияния
слияние ветки	`protocol (2ed5970)`	`capture`	`a5535e0`
слияние в main	`capture`	`main`	`779238c`

Единственный конфликт был в `README.md`, где обе стороны дописали записи в список ссылок документации. Разрешён сохранением обоих списков без удаления чего-либо.

После слияния:

- `pytest tests/ -q` сообщает 385 пройдено (историческое число того слияния; на текущей `main` набор — 957 тестов, 794 на архивном `v1.0.0`, раздел 11).
- `ruff check . --select "E4,E7,E9,F" --ignore E501` сообщает All checks passed.
- Прогон CI 37983669575 зелёный: матрица тестов (3.12 и 3.13 на `ubuntu-latest`, 3.12 на `ubuntu-22.04`), `code-quality` и `dependency-audit` проходят.

11. Финальное состояние

Проверено на слитой ревизии, после того как наборы локали интерфейса и снимков, перцептивное сравнение базовых линий и гигиена репозитория были слиты в `main`.

Пункт	Значение
Вершина `main`	слияние работы по презентации в `main` (`519c585`)
Набор (рабочая копия)	794 пройдено, 0 пропущено
Набор (`submission_*.zip` из релиза v1.0.0, чистая распаковка)	793 пройдено, 1 пропущено (тест бита исполняемости; zip теряет режим)
Линт	`ruff check . --select "E4,E7,E9,F" --ignore E501`: All checks passed
Покрытие	90% по `src/` (`pytest --cov=src`); крупнейший пробел — связывание `src/ui/`
CI	зелёный на `main`: матрица тестов (3.12 и 3.13), матрица `ui`, `code-quality`, `dependency-audit`
Крупнейший отслеживаемый файл	`presentation/screencast.mp4`, 1.9 МиБ, заметно меньше 5 МиБ
Архив сдачи	`submission_20261012_1944.zip`, опубликованный ресурс `v1.0.0`
Чистая распаковка	793 пройдено, 1 пропущено из чистой распаковки этого ресурса

Проверки следов на этой ревизии:

- `git ls-files | grep -iE "cache|assistant"`: нет результата.
- `git log --all -p | grep -iE "ghp_|sk-|api_key"`: только `docs/AUDIT.md`, цитирующий собственную проверку, токена нет.
- Сообщения коммитов: подписи инструмента, модели или ассистента нет.
- `\b(TODO|FIXME|XXX|HACK)\b` в исходниках, тестах и скриптах: нет.

12. Отложено после защиты

Линт-гейт теперь запускает полный набор `ruff`, выбранный в `pyproject.toml` (`E4`, `E7`, `E9`, `F`, `I`, `UP`, `B`, `SIM`, `C4`, `ISC`, `RUF`), как расширено в разделе 15. Две проверки написаны, но закомментированы в рабочем процессе, чтобы их вернуть, когда спадёт давление срока:

Проверка	Почему отложена	План
`ruff format --check .`	100 файлов не отформатированы, ни один в зоне capture/project/report	форматировать зона за зоной, затем в гейт
`mypy .`	114 находок в 13 файлах, ни одна в проверяемой зоне	аннотировать зона за зоной, затем в гейт

Заметки R5

Контракт проекта называет файл `annotations.sqlite`, и `Project.create` его создаёт. `src/project/annotations.py` открывает его лениво и создаёт таблицу `annotations` при первой записи; заметки исследователя на диапазон байт (вид, захват, сессия, направление, смещение, длина, текст) живут в этой таблице с тем же провенансом, что хранит результат правила. `Project.create` хранилище не трогает, поэтому проект без записанных заметок держит корректную пустую базу.

Тайминг тестового стенда

`tests/capture/test_synthetic_stand.py` гоняет `scripts.run_synthetic_stand.run_stand`, который открывает настоящий loopback TCP-сокет и захватывает живой обмен, поэтому границы пакетов зависят от тайминга. Один холодный прогон полного набора упал, и каждый прогон после него проходил. Оставлено как есть до защиты; после неё стенд должен захватывать из закоммиченной фикстуры, а не из живого сокета.

13. Открытые вопросы

- **Захваты организаторов.** Решено. Получены после окна разработки и разобраны: набор закоммичен в `tests/corpus/real/`, конвейер воспроизводит счётчики сессий в `dataset_manifest.json`, а оба направления фреймятся и совпадают на четырёх исследовательских захватах (45 запросов + 45 ответов, 100 % покрытия байт). Наивное чтение оставляет шесть контрпримеров на захвате конфигурации; чтение со смещением заголовка их убирает. Результаты — в `docs/REAL_CORPUS_ANALYSIS.md`; сгенерированный отчёт — `docs/DATASET_INVESTIGATION.md`. Синтетический корпус остаётся для обсуждения фрейминга, поскольку сетевой фрейминг создал организатор.
- **Репетиции защиты.** Решено. У прохода есть спикерский сценарий с маршрутом на 5 и на 10 минут, оба выверены по произносимому тексту (`presentation/defense_script.md`), и чеклист репетиции (`docs/DEFENSE_CHECKLIST.md`).
- **Резервное видео.** Решено. Запись экрана прогона закоммичена как `presentation/screencast.mp4` (3:40, H.264), описана в `presentation/VIDEO.md`.
- **Название группы.** Решено: сдача фиксирует участника и группу `ИС-24`.

14. Историческое авторство

Исторические коммиты несут служебные адреса электронной почты как автора или коммитера. Оставлено на месте (без переписывания). Отображение нормализовано через `.mailmap`.

15. Аудит всего проекта и расширенный линт

Второй проход по всему репозиторию (не только по зоне захвата), начат на `main` на `0466abc` и завершён на `89ada7f` после слияния финальной полировки презентации.

Линт-гейт расширен

Выбор `[tool.ruff]` был расширен со стартового набора до `E`, `F`, `I`, `UP`, `B`, `SIM`, `C4`, `ISC` и остальных групп, отмеченных в `pyproject.toml`. `ruff check .` чист по `src`, `tests`, `scripts` и `presentation`. Находки исправлены автоматически там, где переписывание было безопасным, затем вручную для остальных. Ручные правки, которые стоит назвать:

- `src/protocol/cli.py`: файл, открытый без менеджера контекста (`SIM115`).
- `src/protocol/export.py`: `%`-форматирование заменено на f-строки (`UP031`).
- `tests/hypothesis/test_real_corpus.py`, `tests/protocol/test_cli.py`: тот же образец `SIM115` в тестовых помощниках.
- `tests/integration/test_reference_report.py`: неявная конкатенация строк (`ISC001`).
- `src/report/markdown.py`: имя помощника затеняло встроенную функцию (`A001`).

За одним автоматическим переписыванием пришлось проследить, потому что оно изменило больше, чем форматирование:

- `UP042` превратил `(str, Enum)` в `StrEnum` в `src/hypothesis/status.py` и `src/capture/session.py`. Это заменяет заметку раздела 3, сохранявшую написание `str, Enum`: для публичного кон-

тракта они эквивалентны, потому что каждый писатель сериализует через `.value`, а каждый читатель разбирает через `Status(text)` / `Direction(text)`, никогда через `str(member)`.

IPv6

`src/capture/parser.py` теперь разбирает IPv6 наравне с IPv4. `_layer3_ip` возвращает версию IP вместе с датаграммой; `_parse_tcp` и проверка контрольной суммы диспетчеризуются по ней, а контрольные суммы TCP для IPv6 проверяются по IPv6-псевдозаголовку. TCP-сессия поверх IPv6 даёт поток точно как IPv4. Фрагментация IPv6 теперь собирается: `src/capture/fragment.py` собирает фрагменты датаграммы для обеих версий IP, сообщает `ip_fragment_reassembled`, `ip_fragment_incomplete` и `ip_fragment_overlap` и держит список испускаемых парсером типов диагностики в шаге со схемой.

`docs/CAPTURE_API.md`, `docs/CONTRACT.md`, `ARCHITECTURE.md` и `docs/PROJECT_MAP.md` больше не перечисляют IPv6 как неподдерживаемый. Схема держит `ipv6_ignored` в своём перечислении, чтобы файлы, записанные до этого изменения, всё ещё проходили проверку, но парсер его больше не испускает. `tests/fixtures/ipv6_session.pcapng` и `tests/fixtures/defects/fragmented_ipv6.pcapng` покрывают оба пути; два теста, закреплявших старое поведение «пропускать IPv6», переписаны.

Презентация

Значок воспроизведения на слайде с записью также был вынесен из центра кадра в этом проходе. Тот же фикс сначала попал в `main`, в полировку презентации (`0c164e8`, значок 40 px в правом нижнем углу); они совпадают по размеру и размещению, поэтому этот проход сохраняет ту версию и вносит только нужную согласованность исходников. Переписывание `C408`, описанное выше, — то, чему пришлось научить страж синхронизации презентации.

Финальные числа

Проверка	Результат
`pytest tests/ -q` (LANG=en_US.UTF-8, `QT_QPA_PLATFORM=offscreen`)	957 пройдено, 0 упало
`ruff check .`	All checks passed
`ruff format --check .`	100 файлов были бы переформатированы (всё ещё отложено)
`mypy src/`	114 находок в 13 файлах (всё ещё отложено)
`presentation/slides.pdf`, `slide-light.pdf`	15 страниц, пустой страницы нет

Находки `mypy` сосредоточены в трёх файлах: `src/protocol/engine.py` (75), `src/ui/main_window.py` (17) и `src/protocol/rule.py` (6). Число по движку — в основном один образец: `FieldResult(**kwargs)`, собранный из отображения смешанных типов, а фикс — типизированный вызов конструктора в каждом из шести мест вызова, что оставлено для отдельного изменения, а не вложено в этот проход.

16. Сверка с tshark

Пересборщик — та часть конвейера, которой рецензент меньше всего может доверять на глаз, поэтому его сверили с независимой реализацией. `tshark` 4.4.18 (Wireshark) собирает те же TCP-потоки командой `tshark -r <file> -q -z follow,tcp,raw,<k>`; `scripts/cross_check_tshark.py` запускает обе и сравнивает байты каждого направления сессии.

Команда:

```
python scripts/cross_check_tshark.py $(find tests/corpus/real -name '*.pcap' -o -name
'*.pcapng')
```

Результат на двенадцати закоммиченных исследовательских и диагностических захватах:

Захват	Направлений	Результат
`additional/04_new_observations.pcapng`	1	байт-в-байт
`diagnostics/01_retransmission.pcap`	1	байт-в-байт
`diagnostics/02_reordered.pcapng`	1	байт-в-байт
`diagnostics/03_missing_segment.pcap`	1	идентично, кроме маркера пропуска ниже
`diagnostics/04_snaplen_truncated.pcapng`	1	байт-в-байт
`diagnostics/05_conflicting_overlap.pcap`	1	байт-в-байт
`diagnostics/06_missing_beginning.pcapng`	1	байт-в-байт
`diagnostics/07_checksum_offload.pcap`	1	байт-в-байт
`diagnostics/08_bad_checksum.pcapng`	1	байт-в-байт
`research/01_configuration.pcap`	2	байт-в-байт
`research/02_variations.pcapng`	1	байт-в-байт
`research/03_combined_messages.pcap`	1	байт-в-байт

Проверено тринадцать направлений сессий, ноль необъяснённых расхождений.

Единственное различие и почему оно верно

`diagnostics/03_missing_segment.pcap` имеет дыру в середине направления ответа. `tshark` пишет буквальный маркер `[35 bytes missing in capture file]\x00` в свой вывод `follow` — человекочитаемый заполнитель ровно ширины дыры. Этот проект хранит только наблюдаемые байты и записывает дыру как структурированную диагностику: `gap(offset 0, length 35)`. После удаления маркера из обоих направлений наблюдаемые участки совпадают байт в байт, а удалённая ширина (35) равна записанной длине пропуска. Контракт в `docs/CONTRACT.md` требует ровно этого — `bytes_b64` никогда не содержит синтезированных байтов, — поэтому расхождение — намеренное поведение, а не дефект пересборки. Скрипт сверки распознаёт этот случай и сообщает о нём отдельно, а не считает его несовпадением.

Две политики различаются лишь тем, как представлена дыра, и никогда — байтами, которые потребитель видит для действительно захваченных диапазонов.